Chapter 38 Using Communications

This chapter shows how to use the serial ports for input and output, including use of Husky communications (configurable handshaking, buffering, protocols, etc).

38.1 Standard facilities

Hunter 16 provides two serial ports: COM1, a fully-featured RS232 with modem control lines, and COM2, a simple RS232 port with TxD RxD, Gnd and CTS only.

38.1.1 BIOS

The standard communications facilities, provided by INT 14H, are as follows:

00H Initialise port
01H Send a character
02H Receive a character
03H Get port status
04H Extended initialise port (PS/2-compatible)

These are fully supported. (See Chapter 24.)

For example, to use these facilities to send data, you would typically:

1. Configure the speed and data format to suit the other system (function 0).
2. Poll the status (function 3) until the shift register is empty (bit 6).
3. Send the next character (function 1).
4. Check the status (function 3) to see if the character has transmitted ok and handle the timeout or other error, if not.
5. If there is more data to send, jump back to step 2.

38.1.2 Interrupts

Use of standard (BIOS) communications facilities does not generate any hardware interrupts; the BIOS polls the ports. The hardware may however be configured to generate interrupts, as described in section 32.4.4.

38.1.3 RAM

Use of standard (BIOS) communications facilities does not use any RAM; there is no buffering.

38.1.4 Hardware

Many communications programs ignore the rather primitive BIOS communications support and control the hardware directly, in order to improve performance (for example, to implement higher speeds, protocols, buffering or handle interrupts).

- **Hunter 16** emulates the 8250 device used for serial ports in a standard IBM. Most facilities are implemented, with the following exceptions:
- For COM1, modem status interrupts (other than delta RI) are not generated immediately but after the change in status (by polling). For COM2, only delta CTS is generated. No 5-bit and 6-bit data.
- Parity bit cannot be forced to 1.
- No 1.5 stop bits.

(Most of these limitations can be overcome by fitting a communications expansion board option.)

For more details of the hardware emulation, see section 32.4.4.

38.2 Extra facilities

38.2.1 BIOS

The following extra facilities are provided by INT 14H:

IEH	Read comms parameters
20H	Select comms parameters
21H	Extended control
22H	Extended status
23H	Control handshaking
24H	Control CTS
25H	Control line drivers
26H	Control RI power up
27H	Get installed protocols count
28H	Get protocol name
29H	Get protocol handle
2AH	Extended protocol menu
2BH	Get extended protocol parameters
2CH	Control DTR

You can detect communications errors or receipt of data by using the event-handling mechanism, which is accessed via the following INT 15H functions:

6DH	Get event details
6EH	Enable/trap event
6FH	Acknowledge event

These functions are described in detail in Chapter 25. For more details of events, see Chapter 41.

38.2.2 Husky communications

The majority of the extended facilities are used to configure and control the Husky communications package. This is interrupt-driven and buffered for efficiency, with configurable handshaking and protocols for flexibility and advanced facilities such as automatic error detection.

Once selected, Husky communications replaces the simple IBM communications, and is entirely transparent to standard applications that use INT 14H, functions 1..3 for serial I/O and status checking.

Applications that initialise communications directly (using INT 14H functions 0 or 4) will simply replace Husky communications with their specified configuration. Husky communications will be restored by a cold start (turning on in non-resume mode, or after emergency breakout or loss of all power).

NOTE: Selecting Husky communications changes the serial emulation, so that it will not generate the expected communications port interrupts. This is unlikely to cause problems, as any program that is going to use the interrupts will start off by re-programming the interrupt controller and serial port registers; the **Hunter 16** BIOS will detect this and restore the normal emulation automatically.

See section 38.3.1 for an illustration of using Husky communications.

Part 2 includes extensive background information on communications, including handshaking and protocols, so that will not be repeated here.

38.2.3 Configuration menu

A user can also select and configure Husky communications by using the UTIL utility.

Refer to Part 2 for background information on communications, including handshaking and protocols.

38.2.4 Interface power

An RS232 interface requires significant amounts of power. To conserve battery power, the RS232 interfaces are left turned off when **Hunter 16** is turned on. They turn on automatically when a program attempts to send or receive data, or if the handshake lines become active.

The port then remains turned on until **Hunter 16** is turned off, so further transmissions are not affected in this way. To conserve power, you should turn the port off again, by using INT 14H function 25H.

38.3 Using the extra facilities

38.3.1 Transmitting a file

To transmit a file using Husky communications, you could use the following steps. Where appropriate, the BIOS function required is specified in brackets. Note that you are not concerned with the low level mechanics of handshaking and protocols, once you have configured them as required.

```
;*******************************
;* EXAMPLE: TRANSMITTING A FILE *
;*******************************

START:

;INITIALISE

select and configure communications (INT 14H, 20H)
set n to 0
open input file
get file length
prompt user to connect

; PREVENT INTERRUPTIONS

disable power off (INT 15H, 69H)
disable rebooting (INT 16H, 22H)
warn user cannot reboot/power off/disconnect

;TRANSMIT FILE

LOOP1:
   get character n
   send character (INT 14H, 1)
   check status byte returned
   if bit 7 is 0

     ;CHARACTER PLACED IN BUFFER OK
     increment count
     if count less than length, goto LOOP1
     else goto LOOP2 ;SENT LAST CHARACTER
   else
     ;HANDLE ERRORS
     if bít 7=1 ;TIMEOUT
          display timeout error message
          ask user whether to cont inue
          if answer is wYES" goto LOOP1
          else flush buffer (INT 14N, 21H)
               goto DONE

;WAIT FOR TRANSMISSION TO FINISH
LOOP2:
   get line status (INT 14M, 22H)
   if buffer not empty goto LOOP2
```

```
;TIDY UP

DONE:
tell user can disconnect
turn off interface (INT 14H, 25H)
enable power off (INT 15H, 69H)
enable rebooting (INT 16H, 22H)
return
```

As it stands, this routine simply waits whenever the transmit buffer becomes full, which is very wasteful if there is some other processing that the main program could be doing (particularly at low Baud rates) For example, you could check if the buffer is full (using INT 14H, function 22H) before attempting to write a character to it and return if it is, then check it regularly in the main program and only call the routine again when the buffer becomes empty again.

38.3.2 Receiving a file

To receive a file using Husky communications, you could use the following steps. Where appropriate, the BIOS function required is specified in brackets. Note that you are not concerned with the low level mechanics of handshaking and protocols, once you have configured them as required.

Also note that in this example, a timeout is used to detect the end of transmission, which will work for binary or text files, with simple protocols. If the file is a simple text file, end-of- file might also be indicated by receipt of a Ctrl-Z character (1AH).

```
;******************************
;* EXAMPLE: RECE IVING A FILE *
;******************************

START:

;INITIALISE
select and configure communications (INT 14H, 20H)
open output file
prompt user to connect
```

```
;PREVENT INTERRUPTIONS

disable power off (INT 15H, 69H)
disable rebooting (INT 16H, 22M)
warn user cannot reboot/power off/disconnect

;RECEIVE FILE

LOOP1:
     input character (INT 14H, 2)
     check status byte returned
     if bit 7 is 0
          ;CHARACTER RECEIVED OK
          Write character to file
          goto LOOP1
     else
          ; HANDLE ERRORS/END OF FILE
          If bit 7:1 :TIMEOUT
               display timeout error message
               ask user whether to continue
               if answer is “YES” goto LOOP1
          else goto DONE ;TREAT AS END OF FILE

;TIDY UP

DONE:
close output file
flush Rx buffer (INT 14H, 21H)
tell user can disconnect
turn off interface (INT 14H, 25M)
enable power off (INT 15H, 69H)
enable rebooting (INT 16H, 22H)

return
```

As it stands, this routine simply waits whenever the receive buffer becomes empty, which is very wasteful if there is some other processing that the main program could be doing. To avoid this, the routine could check if the buffer is empty (using INT 14H function 22H) before attempting to read a character from it and return if it is, then the main program could check the buffer regularly and only call the routine again when there is data to ready again. The main program would also have to "time out' if the buffer remains empty for a period, to detect the end of file.

38.3.3 Using an autoanswer modem

An autoanswer modem must alert **Hunter 16** when it receives a call, so that your program can handle the call. It does this by raising the RI (Ringing In) line, which can be detected by monitoring bit 6 of the modem status (using INT 14H function 3), or trapping the RI interrupt (if enabled) as on a desktop. Hunter 16 BIOS provides two extra facilities: event handling and automatic power-up.

If you enable the RI event (INT 15H, function 6EH), the BIOS will remember that RI has been received for you. This has two advantages:

it allows you to monitor this along with any other critical events (if you've enabled them) with a single function call (INT 15H, function 6DH), and if the RI signal is over quickly, you still won't miss it.

Automatic power-up, if enabled, allows you to conserve power by turning **Hunter 16** off! When the call is received, RI will turn **Hunter 16** on so your program can handle the call. When the call is finished, the program can turn off again, waiting for the next call.

The following code shows these techniques in action:

```
;*************************************
;* EXAMPLE: USING AN AUTOANSWER MODEM *
;*************************************

START:

;INITIALISE

select and configure comunications (INT 14H, 20H)
enable RI event (INT 15H, 6EH)
enable RI power up (INT 15H, 27H)

START1:

enable power off (INT 15H, 69H)
turn off (INT 15H, 68H)
```

```
;HAVE TURNED ON. CHECK WHY
get event details (INT 15H, 60H)
if RI bit not set
     ;MUST BE USER POWER ON
     enable reboot (INT 16H, 22H)
     display message saying reboot or leave alone
     pause for 10 seconds     ;USER REBOOTS NOW OR NEVER
     goto START1              ;NO REBOOT SO TURN OFF AGAIN


;RI, SO HANDLE CALL

;PREVENT INTERRUPT IONS

disable power off (INT 15H, 69H)
disable rebooting (INT 16H, 22H)
display message saying answering call

;RECEIVE FILE

set error count to 0
open output file

LOOP1:
   input character (INT 14H, 2)
   check status byte returned
   if bit 7=0
     ;CHARACTER RECEIVED OK
     Write character to file
     goto LOOP 1
   else
     ; HANDLE ERRORS/END OF FILE
     If bit 7=1                          ;TIMEOUT
     goto DONE                           ;TREAT AS END OF FILE

     else                                ;OTHER ERROR
          increment error_count
          if error count < limit
          then goto LOOP1                ;TOLERATE SOME ERRORS
          else                      ;TOO MANY, SO TERMINATE CALL
               drop DTR (INT 144, 23M   ;TURN OFF MODEM
               goto DONE                ;TREAT AS END OF FILE
```

```
;TIDY UP

DONE:
close output file
flush Rx buffer (INT 14M, 21H)
ack RI event (INT 15H, 6FH)   ;SO CAN ANALYSE NEXT POWER UP
OK
goto START 1                  ;TURN OFF, READY FOR NEXT CALL
```